Smart Personal Assistant

Context-Aware Intelligent Agent | Final Project Extension

Course project: Intelligent Agent
Implementation: Python command-line application

1. Problem formulation

The project is a local text assistant for everyday requests. Phase 1 handles greetings, date and time, basic arithmetic, notes, and reminders. The final extension adds day planning, rule-based reasoning, uncertainty-aware clarification, explanations, and user-controlled data deletion. The agent follows **Perceive -> Decide -> Act**: it normalizes a message, selects an intent with a confidence score, applies relevant rules, and responds or takes an action.

The scheduling extension is a constraint satisfaction problem (CSP). Each reminder or meeting is a variable whose value is its start time. Its domain contains 30-minute slots that satisfy task duration, earliest time, deadline, working hours (09:00-17:00), and fixed lunch break (12:00-13:00). Fixed-time meetings have singleton domains. Binary constraints require every pair of tasks to have non-overlapping intervals. The goal test is a complete assignment satisfying every constraint.

2. Approach

Intent recognition uses transparent phrase patterns rather than a hidden model. Each recognized pattern returns a score. Below 0.62, the assistant asks a clarifying question and stores the unanswered query. This score is a rule-based confidence indicator, not a calibrated probability. Missing note or task information starts multi-turn slot filling. Task fields include duration, earliest time, deadline, and fixed start.

The knowledge base uses first-order-style predicates and forward chaining. The engine matches premises to facts, binds variables, appends conclusions, and records a trace. The assistant can show this trace with **why did you do that?**.

Algorithms and analysis

The explicit rules are:

```
Meeting(x) AND StartsBefore(x, "10:00") -> Remind(x, 30)
Task(x) AND DueToday(x) -> Priority(x, "high")
```

The plain CSP solver uses chronological backtracking. The enhanced solver selects the variable with the smallest remaining domain, applies forward checking after each choice, and enforces arc consistency with AC-3. If scheduling fails, the assistant reports empty domains and identifiable fixed-time overlaps.

3. Complexity

Let *n* be task count and *d* the largest domain size. Plain backtracking has worst-case time **O(d^n)** and uses **O(nd)** space for domains and recursion state. Forward checking filters neighboring domains after each assignment but does not improve the worst-case exponential bound. AC-3 has the standard worst-case bound **O(e d^3)** for *e* directed constraint arcs and domain size *d*. Domains and the arc queue use **O(nd + e)** space.

The planner uses fixed 30-minute granularity and one workday. It returns the first valid schedule rather than optimizing preferences. Longer tasks must be multiples of 30 minutes. Travel time, priorities, and task dependencies are not modeled.

4. Results

Three deterministic plans with 3, 5, and 8 tasks were each solved once by both methods. The table shows assignment attempts and wall-clock time from one local run. Runtime varies by computer and Python version.

Plan	Tasks	Plain BT tried	Plain BT seconds	BT + FC + AC-3 tried	BT + FC + AC-3 seconds
Small	3	7	0.0000614	3	0.0001226
Medium	5	19	0.0000643	5	0.0002950
Large	8	47	0.0001145	8	0.0008583

Both solvers found schedules for all plans. The enhanced solver tried fewer values through constrained-variable selection and propagation. Its runtime was higher on these tiny cases because propagation adds overhead; fewer assignments do not guarantee lower runtime for small instances. Larger or tighter plans are needed to see where pruning pays off.

Evidence and explainability

Inference trace example

Given the facts:

```
Meeting("Project review")
StartsBefore("Project review", "10:00")
```

The forward-chaining engine matches both premises of **early_meeting_reminder**, binds ?x to Project review, and infers:

```
Remind("Project review", 30)
```

The recorded trace includes both source facts, the rule name, the inferred fact, and its explanation that meetings before 10:00 receive a 30-minute advance reminder.

Example shown to the user

After **plan my day**, the assistant presents scheduled tasks and assignments tried. If the user asks **why did you do that?**, it prints the perceived text, selected intent and confidence, and action. When a reminder rule fires, the trace identifies the meeting facts behind that recommendation.

Agent loop and action mapping

Perceive	Decide	Act
Normalize user text	Select intent and confidence	Answer, clarify, store, explain, or schedule

Reproducibility

The comparison is available in the running application with the **benchmark plans** command. The three inputs are deterministic and require no external dataset. Runtime is machine-dependent; rerun the command from a fresh clone. User data is stored locally and excluded from Git.

The existing Phase-1 tests remain in the repository. The extension is not claimed to be covered by those tests; its required comparison and inference example are available through the application and this report.

5. Responsible AI and reflection

Privacy

Notes, tasks, and unanswered query text are stored in a local JSON file without encryption. Queries can contain personal information. Users should avoid sensitive content and clear the data with **forget my data** before sharing a device or repository. The live data file is excluded from Git. The command clears notes, reminders, and query history together.

Bias and fairness

The planner uses user-supplied time windows and does not use demographic attributes. Default working hours and a fixed lunch break may not suit every schedule, culture, care responsibility, accessibility need, or workplace. Users can enter earliest, deadline, and fixed-time constraints. Future work should make hours and breaks configurable and invite users to review recommendations before acting.

Accountability and governance

The assistant does not send reminders externally or take actions without the user. It presents schedules for review, explains rule conclusions, and logs unanswered requests for improvement. Query logs should be reviewed and deleted when no longer needed. Before use beyond a classroom demo, data handling, retention, access controls, and applicable institutional and privacy requirements should be reviewed. The local lookup is limited; the assistant is not a source of professional advice.

Reflection

The main design challenge is balancing search reduction against propagation overhead while keeping the CSP understandable. The enhanced solver reduced assignments in the sample plans, but propagation overhead made it slower at these small sizes. Next steps include automated tests for unsatisfiable schedules, configurable work hours, improved date/time parsing, and a visual schedule.

Conclusion

The project demonstrates the intelligent-agent loop, explicit rules, forward-chaining inference, confidence-based clarification, explainable actions, user-controlled data deletion, and measurable CSP comparison. AI assistance was used during development. The student should review and be prepared to explain every component and adapt it during the viva.

References

Python Software Foundation, *Python 3 Documentation*, https://docs.python.org/3/
Mackworth, A. K. (1977), “Consistency in Networks of Relations,” *Artificial Intelligence*, 8(1), 99-118.
Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*, 4th edition, Pearson, 2021.